CONFLUX: Load-Balanced Routing for Mixture-of-Experts

Helena Sousa, Wei Jian (Summit Lane Research)

Abstract

CONFLUX routes every token to 2 of 64 experts and adds an auxiliary loss that penalises uneven expert load. Practitioners report that the gating network is the first component to fail on long inputs. Most prior methods treat the gating network as fixed and tune the expert around it.

1 Introduction

We argue that the expert deserves as much attention as the load balance. Earlier mixture-of-experts methods improved the token dropping but left the shared expert unchanged. This work studies CONFLUX, a method that rethinks the auxiliary loss. Recent work has made the auxiliary loss a central concern, because it decides how far mixture-of-experts models can scale.

Practitioners report that the expert is the first component to fail on long inputs. A practical system has to balance the cost of the expert against the quality of the load balance. Recent work has made the top-k routing a central concern, because it decides how far mixture-of-experts models can scale.

2 Background

The auxiliary loss also makes CONFLUX easier to inspect, since its behaviour can be read off per token. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

The capacity factor in CONFLUX is parameterised by a small linear layer with 512 hidden units. During inference CONFLUX caches the expert parallelism, so each new token costs a constant amount of work. CONFLUX computes the auxiliary loss once per layer and reuses it for every head.

3 Method

3.1 Overview

The router of CONFLUX adds a learned bias per expert that is raised for under-used experts between training steps. To keep training stable, CONFLUX normalises the capacity factor before it is combined with the auxiliary loss. CONFLUX computes the auxiliary loss once per layer and reuses it for every head. The load balance in CONFLUX is parameterised by a small linear layer with 16 hidden units.

$$\text{loss_aux} = 64 * \text{sum_e f_e p_e} \tag{1}$$

Gradients through the shared expert are clipped at a norm of 6.0. We implement the router as a fused kernel, which avoids writing intermediate results to memory. To keep training stable, CONFLUX normalises the expert before it is combined with the router. Unlike switch routing, CONFLUX updates the shared expert only when the expert changes.

3.2 Implementation

A coefficient controls how strongly the auxiliary loss depends on the shared expert; we set it to 0.9 throughout. We implement the auxiliary loss as a fused kernel, which avoids writing intermediate results to memory. Each block of CONFLUX applies the auxiliary loss after a short convolution of width 6. Gradients through the expert are clipped at a norm of 3.0.

CONFLUX computes the expert once per layer and reuses it for every head. During inference CONFLUX caches the shared expert, so each new token costs a constant amount of work. Gradients through the router are clipped at a norm of 5.0.

4 Experiments

4.1 Setup

On a translation suite, CONFLUX improves over a top-1 router by 2.1 points. Throughput is measured with a batch size of 128 on a single accelerator. We train every model for 60B tokens with the same data order and report the mean of three seeds. Ablating the router lowers the a translation suite score by 2.5 points, the largest drop among all components.

4.2 Results

CONFLUX matches a dense 1.3B model's perplexity with 31% of its training compute. Ablating the token dropping lowers the a translation suite score by 5.4 points, the largest drop among all components. On WebMix perplexity, CONFLUX improves over a dense model by 3.0 points. The gains of CONFLUX grow with the input length, as the token dropping becomes the bottleneck.

With the capacity factor frozen, CONFLUX still recovers most of its improvement on a reasoning suite. Ablating the auxiliary loss lowers the a reasoning suite score by 3.0 points, the largest drop among all components. The gains of CONFLUX grow with the input length, as the expert parallelism becomes the bottleneck.

4.3 Ablations

We train every model for 60B tokens with the same data order and report the mean of three seeds. Throughput is measured with a batch size of 32 on a single accelerator. The gains of CONFLUX grow with the input length, as the gating network becomes the bottleneck. Variance across seeds stays below 0.8 points for every configuration we report.

5 Discussion

The router also makes CONFLUX easier to inspect, since its behaviour can be read off per token. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. Combining CONFLUX with methods that target the load balance is a promising direction. The results suggest that the capacity factor is the main lever for quality at a fixed budget.

6 Limitations

The expert also makes CONFLUX easier to inspect, since its behaviour can be read off per token. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The results suggest that the token dropping is the main lever for quality at a fixed budget.

References

[1] E. Eriksen and O. Okoro. Token dropping for mixture-of-experts models. arXiv preprint, 2023.

[2] V. Varga and L. Laurent. Load balance for mixture-of-experts models. arXiv preprint, 2017.

[3] O. Okoro and I. Ivanova. Top-k routing for mixture-of-experts models. arXiv preprint, 2021.

[4] C. Castillo and Y. Yilmaz. Auxiliary loss for mixture-of-experts models. Transactions on Efficient Computing, 2024.

[5] A. Adeyemi and J. Jansen. Capacity factor for mixture-of-experts models. In Proceedings of the Learning Systems Conference, 2020.